

You Only Look Once: Unified, Real-Time Object Detection

Joseph Redmon*, Santosh Divvala*[†], Ross Girshick[¶], Ali Farhadi*[†]

University of Washington*, Allen Institute for AI[†], Facebook AI Research[¶]

<http://pjreddie.com/yolo/>

Abstract

We present YOLO, a new approach to object detection. Prior work on object detection repurposes classifiers to perform detection. Instead, we frame object detection as a regression problem to spatially separated bounding boxes and associated class probabilities. A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation. Since the whole detection pipeline is a single network, it can be optimized end-to-end directly on detection performance.

Our unified architecture is extremely fast. Our base YOLO model processes images in real-time at 45 frames per second. A smaller version of the network, Fast YOLO, processes an astounding 155 frames per second while still achieving double the mAP of other real-time detectors. Compared to state-of-the-art detection systems, YOLO makes more localization errors but is less likely to predict false positives on background. Finally, YOLO learns very general representations of objects. It outperforms other detection methods, including DPM and R-CNN, when generalizing from natural images to other domains like artwork.

1. Introduction

Humans glance at an image and instantly know what objects are in the image, where they are, and how they interact. The human visual system is fast and accurate, allowing us to perform complex tasks like driving with little conscious thought. Fast, accurate algorithms for object detection would allow computers to drive cars without specialized sensors, enable assistive devices to convey real-time scene information to human users, and unlock the potential for general purpose, responsive robotic systems.

Current detection systems repurpose classifiers to perform detection. To detect an object, these systems take a classifier for that object and evaluate it at various locations and scales in a test image. Systems like deformable parts models (DPM) use a sliding window approach where the classifier is run at evenly spaced locations over the entire image [10].

More recent approaches like R-CNN use region proposal

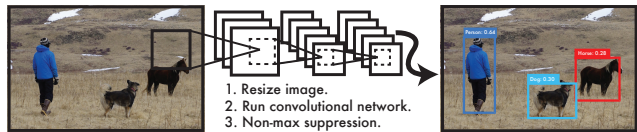


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model’s confidence.

methods to first generate potential bounding boxes in an image and then run a classifier on these proposed boxes. After classification, post-processing is used to refine the bounding boxes, eliminate duplicate detections, and rescore the boxes based on other objects in the scene [13]. These complex pipelines are slow and hard to optimize because each individual component must be trained separately.

We reframe object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. Using our system, you only look once (YOLO) at an image to predict what objects are present and where they are.

YOLO is refreshingly simple: see Figure 1. A single convolutional network simultaneously predicts multiple bounding boxes and class probabilities for those boxes. YOLO trains on full images and directly optimizes detection performance. This unified model has several benefits over traditional methods of object detection.

First, YOLO is extremely fast. Since we frame detection as a regression problem we don’t need a complex pipeline. We simply run our neural network on a new image at test time to predict detections. Our base network runs at 45 frames per second with no batch processing on a Titan X GPU and a fast version runs at more than 150 fps. This means we can process streaming video in real-time with less than 25 milliseconds of latency. Furthermore, YOLO achieves more than twice the mean average precision of other real-time systems. For a demo of our system running in real-time on a webcam please see our project webpage:

<http://pjreddie.com/yolo/>.

Second, YOLO reasons globally about the image when

6
1
0
2
y
a
M
9
J
V
C
s
c
l
5
v
0
4
6
2
0
6
0
5
1
v
i
X
r
a

你只看一次：统一的实时目标检测

约瑟夫·雷德蒙*、桑托什·迪瓦拉*†、罗斯·吉斯克¶、阿里·法哈迪*† University of Washington*, Allen Institute for AI†, Facebook AI Research¶ <http://pjreddie.com/yolo/>

摘要

We present YOLO, a new approach to object detection. Prior work on object detection repurposes classifiers to perform detection. Instead, we frame object detection as a regression problem to spatially separated bounding boxes and associated class probabilities. A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation. Since the whole detection pipeline is a single network, it can be optimized end-to-end directly on detection performance.

Our unified architecture is extremely fast. Our base YOLO model processes images in real-time at 45 frames per second. A smaller version of the network, Fast YOLO, processes an astounding 155 frames per second while still achieving double the mAP of other real-time detectors. Compared to state-of-the-art detection systems, YOLO makes more localization errors but is less likely to predict false positives on background. Finally, YOLO learns very general representations of objects. It outperforms other detection methods, including DPM and R-CNN, when generalizing from natural images to other domains like artwork.

1. 引言

人类瞥一眼图像，便能瞬间知晓其中包含哪些物体、它们的位置以及如何互动。人类的视觉系统快速而精准，使我们能在几乎无需刻意思考的情况下完成驾驶等复杂任务。快速、准确的目标检测算法将使计算机无需专用传感器即可驾驶汽车，助力辅助设备向人类用户传递实时场景信息，并释放通用响应式机器人系统的潜力。

当前的检测系统将分类器重新用于执行检测任务。为了检测一个物体，这些系统会采用针对该物体的分类器，并在测试图像的不同位置和尺度上进行评估。例如，可变形部件模型（DPM）等系统采用滑动窗口方法，在整个图像上以均匀间隔的位置运行分类器[10]。

更近期的如R-CNN等方法采用区域提议

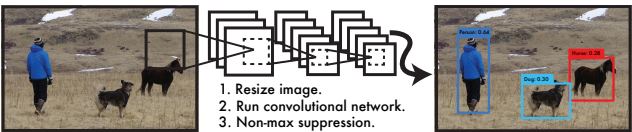


图1：YOLO检测系统。使用YOLO处理图像简单直接。我们的系统（1）将输入图像尺寸调整为448×448，（2）在图像上运行单一卷积网络，（3）根据模型置信度对检测结果进行阈值处理。

方法首先在图像中生成潜在的边界框，然后在这些提议框上运行分类器。分类后，通过后处理来优化边界框、消除重复检测，并根据场景中的其他物体重新评分这些框[13]。这些复杂的流程速度慢且难以优化，因为每个单独的组件都必须单独训练。

我们将目标检测重新定义为一个单一的回归问题，直接从图像像素到边界框坐标和类别概率。使用我们的系统，只需对图像看一眼（YOLO），就能预测出存在哪些物体以及它们的位置。

YOLO令人耳目一新的简洁性如图1所示：单个卷积网络同时预测多个边界框及其类别概率。YOLO直接在完整图像上进行训练，并直接优化检测性能。相比传统目标检测方法，这种统一模型具有多项优势。

首先，YOLO的速度极快。由于我们将检测任务构建为回归问题，因此无需复杂的处理流程。在测试时，我们只需对新图像运行神经网络即可预测检测结果。我们的基础网络在Titan X GPU上无需批处理即可达到每秒45帧，快速版本更可超过150帧/秒。这意味着我们能够以低于25毫秒的延迟实时处理流媒体视频。此外，YOLO的平均精度均值达到其他实时系统的两倍以上。如需查看我们通过摄像头实时运行系统的演示，请访问项目网页：<http://pjreddie.com/yolo/>。

其次，YOLO在预测时会对图像进行全局推理，当

making predictions. Unlike sliding window and region proposal-based techniques, YOLO sees the entire image during training and test time so it implicitly encodes contextual information about classes as well as their appearance. Fast R-CNN, a top detection method [14], mistakes background patches in an image for objects because it can't see the larger context. YOLO makes less than half the number of background errors compared to Fast R-CNN.

Third, YOLO learns generalizable representations of objects. When trained on natural images and tested on artwork, YOLO outperforms top detection methods like DPM and R-CNN by a wide margin. Since YOLO is highly generalizable it is less likely to break down when applied to new domains or unexpected inputs.

YOLO still lags behind state-of-the-art detection systems in accuracy. While it can quickly identify objects in images it struggles to precisely localize some objects, especially small ones. We examine these tradeoffs further in our experiments.

All of our training and testing code is open source. A variety of pretrained models are also available to download.

2. Unified Detection

We unify the separate components of object detection into a single neural network. Our network uses features from the entire image to predict each bounding box. It also predicts all bounding boxes across all classes for an image simultaneously. This means our network reasons globally about the full image and all the objects in the image. The YOLO design enables end-to-end training and real-time speeds while maintaining high average precision.

Our system divides the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that grid cell is responsible for detecting that object.

Each grid cell predicts B bounding boxes and confidence scores for those boxes. These confidence scores reflect how confident the model is that the box contains an object and also how accurate it thinks the box is that it predicts. Formally we define confidence as $\Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$. If no object exists in that cell, the confidence scores should be zero. Otherwise we want the confidence score to equal the intersection over union (IOU) between the predicted box and the ground truth.

Each bounding box consists of 5 predictions: x, y, w, h , and confidence. The (x, y) coordinates represent the center of the box relative to the bounds of the grid cell. The width and height are predicted relative to the whole image. Finally the confidence prediction represents the IOU between the predicted box and any ground truth box.

Each grid cell also predicts C conditional class probabilities, $\Pr(\text{Class}_i | \text{Object})$. These probabilities are conditioned on the grid cell containing an object. We only predict

one set of class probabilities per grid cell, regardless of the number of boxes B .

At test time we multiply the conditional class probabilities and the individual box confidence predictions,

$$\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}} \quad (1)$$

which gives us class-specific confidence scores for each box. These scores encode both the probability of that class appearing in the box and how well the predicted box fits the object.

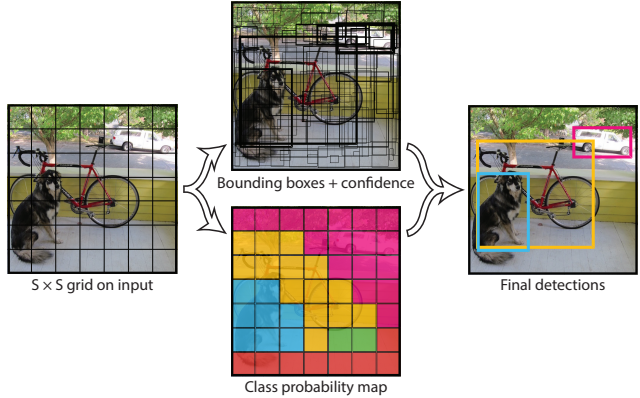


Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

For evaluating YOLO on PASCAL VOC, we use $S = 7$, $B = 2$. PASCAL VOC has 20 labelled classes so $C = 20$. Our final prediction is a $7 \times 7 \times 30$ tensor.

2.1. Network Design

We implement this model as a convolutional neural network and evaluate it on the PASCAL VOC detection dataset [9]. The initial convolutional layers of the network extract features from the image while the fully connected layers predict the output probabilities and coordinates.

Our network architecture is inspired by the GoogLeNet model for image classification [34]. Our network has 24 convolutional layers followed by 2 fully connected layers. Instead of the inception modules used by GoogLeNet, we simply use 1×1 reduction layers followed by 3×3 convolutional layers, similar to Lin et al [22]. The full network is shown in Figure 3.

We also train a fast version of YOLO designed to push the boundaries of fast object detection. Fast YOLO uses a neural network with fewer convolutional layers (9 instead of 24) and fewer filters in those layers. Other than the size of the network, all training and testing parameters are the same between YOLO and Fast YOLO.

进行预测。与基于滑动窗口和区域提议的技术不同，YOLO在训练和测试时能看到整个图像，因此它隐式地编码了类别的上下文信息及其外观。Fast R-CNN作为一种顶尖的检测方法[14]，由于无法看到更大的上下文，会将图像中的背景区域误判为物体。与Fast R-CNN相比，YOLO的背景错误数量减少了一半以上。

第三，YOLO学习到物体的一般化表示。当在自然图像上训练并在艺术作品上测试时，YOLO以显著优势超越DPM和R-CNN等顶级检测方法。由于YOLO具有高度泛化能力，在应用于新领域或意外输入时更不易失效。

YOLO在准确性方面仍落后于最先进的检测系统。尽管它能快速识别图像中的物体，但在精确定位某些物体（尤其是小型物体）时仍存在困难。我们通过实验进一步探讨了这些权衡关系。

我们的训练和测试代码均为开源。多种预训练模型也可供下载。

2. 统一检测

我们将物体检测的各个独立组件统一到一个单一的神经网络中。我们的网络利用整张图像的特征来预测每个边界框，并同时预测图像中所有类别的所有边界框。这意味着我们的网络能够全局性地推理整张图像及其中所有物体。YOLO的设计实现了端到端的训练和实时速度，同时保持了较高的平均精度。

我们的系统将输入图像划分为一个 $S \times S$ 网格。如果某个物体的中心落入一个网格单元，则该网格单元负责检测该物体。

每个网格单元预测 B 个边界框以及这些框的置信度分数。这些置信度分数反映了模型对于框内包含物体的确信程度，同时也体现了模型对其预测框准确性的评估。形式上，我们将置信度定义为 $\Pr(\text{物体}) * \text{IOU}_{\text{pred}}^{\text{truth}}$ 。如果该单元格中不存在物体，则置信度分数应为零。否则，我们希望置信度分数等于预测框与真实框之间的交并比（IOU）。

每个边界框包含5个预测值： x 、 y 、 w 、 h 以及置信度。 (x, y) 坐标表示边界框中心相对于网格单元边界的偏移量，宽度和高度的预测则相对于整张图像。最后的置信度预测表示预测框与任意真实标注框之间的交并比（IOU）。

每个网格单元还会预测 C 个条件类别概率， $\Pr(\text{类别}_i | \text{对象})$ 。这些概率以网格单元包含对象为条件。我们仅预测

每个网格单元对应一组类别概率，无论边界框数量 B 如何。

在测试时，我们将条件类别概率与各个边界框的置信度预测相乘，

$$\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}} \quad (1)$$

这为我们提供了每个边界框的类别特定置信度分数。这些分数既编码了该类出现在框中的概率，也编码了预测框与对象的匹配程度。

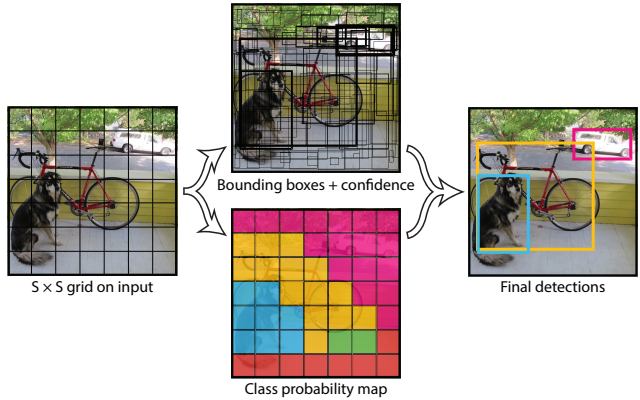


图2：模型。我们的系统将检测建模为一个回归问题。它将图像划分为一个 $S \times S$ 网格，并为每个网格单元预测 B 个边界框、这些框的置信度以及 C 个类别概率。这些预测被编码为一个 $S \times S \times (B * 5 + C)$ 张量。

在PASCAL VOC上评估YOLO时，我们使用 $S = 7$ 、 $B = 2$ 。PASCAL VOC有20个标注类别，因此 $C = 20$ 。我们的最终预测是一个 $7 \times 7 \times 30$ 的张量。

2.1. 网络设计

我们将该模型实现为一个卷积神经网络，并在PASCAL VOC检测数据集[9]上进行了评估。网络的初始卷积层从图像中提取特征，而全连接层则预测输出概率和坐标。

我们的网络架构受到用于图像分类的GoogLeNet模型[34]的启发。我们的网络包含24个卷积层，随后是2个全连接层。我们没有采用GoogLeNet中使用的inception模块，而是简单地使用了1个 1×1 的降维层，后接3个 3×3 的卷积层，这与Lin等人[22]的方法类似。完整的网络结构如图3所示。

我们还训练了一个快速版本的YOLO，旨在突破快速目标检测的界限。Fast YOLO使用了一个卷积层较少（9层而非24层）且各层滤波器数量更少的神经网络。除了网络规模外，YOLO与Fast YOLO的所有训练和测试参数均保持一致。

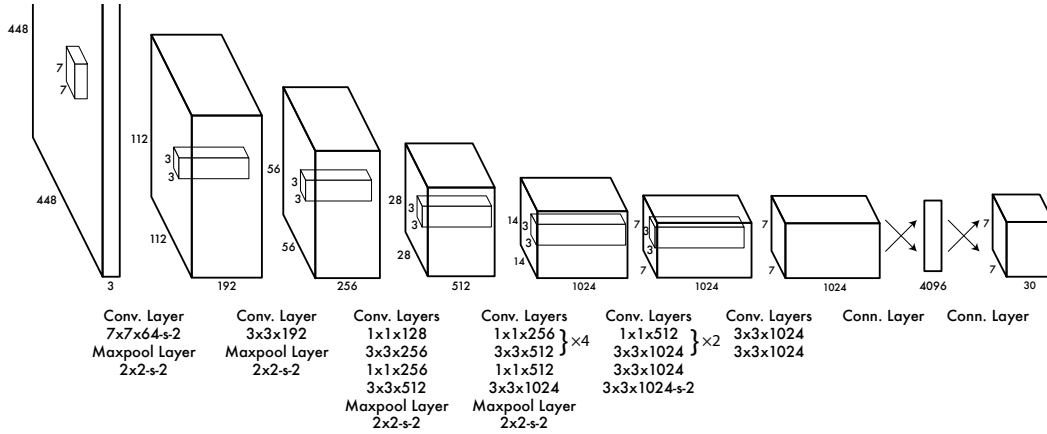


图3: 架构。我们的检测网络包含24个卷积层和2个全连接层。交替的1×1卷积层减少了来自前一层的特征空间。我们首先在ImageNet分类任务上以一半分辨率（224×224输入图像）对卷积层进行预训练，随后将分辨率加倍用于检测任务。

我们网络的最终输出是预测的 $7 \times 7 \times 30$ 张量。

2.2. 训练

我们在ImageNet 1000类竞赛数据集[30]上对卷积层进行了预训练。预训练时，我们采用图3中的前20个卷积层，后接一个平均池化层和一个全连接层。该网络训练耗时约一周，在ImageNet 2012验证集上实现了单次裁剪top-5准确率88%的性能，与Caffe模型库[24]中的GoogLeNet模型表现相当。所有训练和推理均基于Darknet框架[26]完成。

随后，我们将模型转换为用于检测任务。Ren等人研究表明，在预训练网络中同时添加卷积层和全连接层能够提升性能[29]。遵循他们的方法，我们增加了四个卷积层和两个权重随机初始化的全连接层。检测任务通常需要细粒度的视觉信息，因此我们将网络的输入分辨率从224×224提升至448×448。

我们的最后一层同时预测类别概率和边界框坐标。我们将边界框的宽度和高度除以图像的宽度和高度进行归一化，使其值落在0到1之间。我们将边界框 x 和 y 的坐标参数化为特定网格单元位置的偏移量，因此它们也被限制在0到1之间。

我们对最后一层使用线性激活函数，而所有其他层则采用以下带泄漏的修正线性激活函数：

$$\phi(x) = \begin{cases} x, & \text{if } x > 0 \\ 0.1x, & \text{otherwise} \end{cases} \quad (2)$$

我们针对输出中的平方和误差进行优化

模型。我们使用平方和误差是因为它易于优化，但它并不完全符合我们最大化平均精度的目标。它将定位误差与分类误差同等加权，这可能并不理想。此外，在每张图像中，许多网格单元不包含任何物体。这会使这些单元的“置信度”分数趋近于零，常常压倒包含物体的单元产生的梯度。这可能导致模型不稳定，使训练在早期就出现发散。

为了解决这个问题，我们增加了边界框坐标预测的损失，并减少了不包含物体的框的置信度预测损失。我们使用两个参数 λ_{coord} 和 λ_{noobj} 来实现这一点。我们设定 $\lambda_{\text{coord}} = 5$ 和 $\lambda_{\text{noobj}} = .5$ 。

平方误差同样对大框和小框中的误差给予同等权重。我们的误差度量应反映出大框中的小偏差比小框中的小偏差影响更小。为了部分解决这个问题，我们预测边界框宽度和高度的平方根，而不是直接预测宽度和高度。

YOLO为每个网格单元预测多个边界框。在训练时，我们希望每个物体仅由一个边界框预测器负责。我们会根据当前预测值与真实标注框的最高IOU，指定一个预测器“负责”预测该物体。这种做法促使边界框预测器之间形成专业化分工。每个预测器在预测特定尺寸、宽高比或物体类别方面会越来越擅长，从而提升整体召回率。

在训练过程中，我们优化以下多部分

loss function:

$$\begin{aligned}
\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} & \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} & \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} & (C_i - \hat{C}_i)^2 \\
+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} & (C_i - \hat{C}_i)^2 \\
+ \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} & (p_i(c) - \hat{p}_i(c))^2 \quad (3)
\end{aligned}$$

where $\mathbb{1}_i^{\text{obj}}$ denotes if object appears in cell i and $\mathbb{1}_{ij}^{\text{obj}}$ denotes that the j th bounding box predictor in cell i is “responsible” for that prediction.

Note that the loss function only penalizes classification error if an object is present in that grid cell (hence the conditional class probability discussed earlier). It also only penalizes bounding box coordinate error if that predictor is “responsible” for the ground truth box (i.e. has the highest IOU of any predictor in that grid cell).

We train the network for about 135 epochs on the training and validation data sets from PASCAL VOC 2007 and 2012. When testing on 2012 we also include the VOC 2007 test data for training. Throughout training we use a batch size of 64, a momentum of 0.9 and a decay of 0.0005.

Our learning rate schedule is as follows: For the first epochs we slowly raise the learning rate from 10^{-3} to 10^{-2} . If we start at a high learning rate our model often diverges due to unstable gradients. We continue training with 10^{-2} for 75 epochs, then 10^{-3} for 30 epochs, and finally 10^{-4} for 30 epochs.

To avoid overfitting we use dropout and extensive data augmentation. A dropout layer with rate = .5 after the first connected layer prevents co-adaptation between layers [18]. For data augmentation we introduce random scaling and translations of up to 20% of the original image size. We also randomly adjust the exposure and saturation of the image by up to a factor of 1.5 in the HSV color space.

2.3. Inference

Just like in training, predicting detections for a test image only requires one network evaluation. On PASCAL VOC the network predicts 98 bounding boxes per image and class probabilities for each box. YOLO is extremely fast at test time since it only requires a single network evaluation, unlike classifier-based methods.

The grid design enforces spatial diversity in the bounding box predictions. Often it is clear which grid cell an object falls in to and the network only predicts one box for each object. However, some large objects or objects near

the border of multiple cells can be well localized by multiple cells. Non-maximal suppression can be used to fix these multiple detections. While not critical to performance as it is for R-CNN or DPM, non-maximal suppression adds 2-3% in mAP.

2.4. Limitations of YOLO

YOLO imposes strong spatial constraints on bounding box predictions since each grid cell only predicts two boxes and can only have one class. This spatial constraint limits the number of nearby objects that our model can predict. Our model struggles with small objects that appear in groups, such as flocks of birds.

Since our model learns to predict bounding boxes from data, it struggles to generalize to objects in new or unusual aspect ratios or configurations. Our model also uses relatively coarse features for predicting bounding boxes since our architecture has multiple downsampling layers from the input image.

Finally, while we train on a loss function that approximates detection performance, our loss function treats errors the same in small bounding boxes versus large bounding boxes. A small error in a large box is generally benign but a small error in a small box has a much greater effect on IOU. Our main source of error is incorrect localizations.

3. Comparison to Other Detection Systems

Object detection is a core problem in computer vision. Detection pipelines generally start by extracting a set of robust features from input images (Haar [25], SIFT [23], HOG [4], convolutional features [6]). Then, classifiers [36, 21, 13, 10] or localizers [1, 32] are used to identify objects in the feature space. These classifiers or localizers are run either in sliding window fashion over the whole image or on some subset of regions in the image [35, 15, 39]. We compare the YOLO detection system to several top detection frameworks, highlighting key similarities and differences.

Deformable parts models. Deformable parts models (DPM) use a sliding window approach to object detection [10]. DPM uses a disjoint pipeline to extract static features, classify regions, predict bounding boxes for high scoring regions, etc. Our system replaces all of these disparate parts with a single convolutional neural network. The network performs feature extraction, bounding box prediction, non-maximal suppression, and contextual reasoning all concurrently. Instead of static features, the network trains the features in-line and optimizes them for the detection task. Our unified architecture leads to a faster, more accurate model than DPM.

R-CNN. R-CNN and its variants use region proposals instead of sliding windows to find objects in images. Selective

loss function:

$$\begin{aligned} \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (3) \end{aligned}$$

其中 $\mathbb{1}_i^{\text{obj}}$ 表示物体是否出现在单元格 i 中，而 $\mathbb{1}_{ij}^{\text{obj}}$ 表示单元格 i 中的第 j 个边界框预测器对该预测“负责”。

注意，损失函数仅在该网格单元中存在物体时才对分类错误进行惩罚（因此有了之前讨论的条件类别概率）。同时，它也只在该预测器对真实边界框“负责”时（即在该网格单元的所有预测器中具有最高的IOU），才对边界框坐标误差进行惩罚。

我们在PASCAL VOC 2007和2012的训练集与验证集上对网络进行了约135个周期的训练。在针对2012年数据进行测试时，我们也同时将VOC 2007的测试数据纳入训练。整个训练过程中，我们使用的批量大小为64，动量为0.9，衰减率为0.0005。

我们的学习率调度如下：在最初的几个周期中，我们将学习率从 10^{-3} 缓慢提升至 10^{-2} 。如果一开始就使用较高的学习率，模型常会因梯度不稳定而发散。随后，我们以 10^{-2} 继续训练75个周期，接着用 10^{-3} 训练30个周期，最后以 10^{-4} 再训练30个周期。

为避免过拟合，我们采用了dropout和大量数据增强技术。在第一个全连接层后设置丢弃率为0.5的dropout层，以防止层间协同适应[18]。数据增强方面，我们引入随机缩放和平移，幅度最高可达原始图像尺寸的20%。同时，在HSV色彩空间中随机调整图像的曝光度与饱和度，调整系数最高达1.5倍。

2.3. 推理

与训练时一样，预测测试图像的检测结果仅需一次网络评估。在PASCAL VOC数据集上，网络会为每张图像预测98个边界框及每个框的类别概率。YOLO在测试阶段速度极快，因为它仅需单次网络评估，这与基于分类器的方法截然不同。

网格设计在边界框预测中强制实施空间多样性。通常，一个物体落入哪个网格单元是明确的，网络仅为每个物体预测一个边界框。然而，一些大型物体或靠近

多个单元格的边界可以通过多个单元格精确定位。非极大值抑制可用于修正这些多重检测。虽然对性能的影响不如R-CNN或DPM那样关键，但非极大值抑制仍能提升2-3%的mAP。

2.4. YOLO的局限性

YOLO对边界框预测施加了强烈的空间约束，因为每个网格单元仅预测两个边界框且只能对应一个类别。这种空间限制制约了模型能够预测的邻近物体数量。我们的模型在处理成群出现的小物体时表现不佳，例如鸟群。

由于我们的模型从数据中学习预测边界框，因此难以泛化到具有新颖或非寻常宽高比或配置的物体上。我们的模型在预测边界框时还使用了相对粗糙的特征，因为我们的架构从输入图像开始包含多个下采样层。

最后，尽管我们训练时使用的损失函数近似于检测性能，但我们的损失函数在处理小边界框与大边界框中的误差时一视同仁。大边界框中的小误差通常影响不大，但小边界框中的小误差对IOU的影响则要大多。我们误差的主要来源是不准确的定位。

3. 与其他检测系统的比较

目标检测是计算机视觉中的一个核心问题。检测流程通常从输入图像中提取一组鲁棒特征开始（Haar [25]、SIFT [23]、HOG [4]、卷积特征 [6]）。随后，使用分类器 [36, 21, 13, 10] 或定位器 [1, 32] 在特征空间中识别物体。这些分类器或定位器以滑动窗口方式在整个图像上运行，或在图像的某些区域子集上运行 [35, 15, 39]。我们将YOLO检测系统与几种顶尖检测框架进行比较，重点突出其关键相似点与差异。

可变形部件模型。可变形部件模型（DPM）采用滑动窗口方法进行目标检测[10]。DPM使用独立的流程来提取静态特征、分类区域、为高分区域预测边界框等。我们的系统用一个单一的卷积神经网络取代了所有这些不同的部分。该网络同时执行特征提取、边界框预测、非极大值抑制和上下文推理。网络不是使用静态特征，而是在线训练特征并针对检测任务进行优化。我们的统一架构相比DPM实现了更快、更准确的模型。

R-CNN。R-CNN及其变体使用区域提议而非滑动窗口来在图像中查找物体。选择性

Search [35] generates potential bounding boxes, a convolutional network extracts features, an SVM scores the boxes, a linear model adjusts the bounding boxes, and non-max suppression eliminates duplicate detections. Each stage of this complex pipeline must be precisely tuned independently and the resulting system is very slow, taking more than 40 seconds per image at test time [14].

YOLO shares some similarities with R-CNN. Each grid cell proposes potential bounding boxes and scores those boxes using convolutional features. However, our system puts spatial constraints on the grid cell proposals which helps mitigate multiple detections of the same object. Our system also proposes far fewer bounding boxes, only 98 per image compared to about 2000 from Selective Search. Finally, our system combines these individual components into a single, jointly optimized model.

Other Fast Detectors Fast and Faster R-CNN focus on speeding up the R-CNN framework by sharing computation and using neural networks to propose regions instead of Selective Search [14] [28]. While they offer speed and accuracy improvements over R-CNN, both still fall short of real-time performance.

Many research efforts focus on speeding up the DPM pipeline [31] [38] [5]. They speed up HOG computation, use cascades, and push computation to GPUs. However, only 30Hz DPM [31] actually runs in real-time.

Instead of trying to optimize individual components of a large detection pipeline, YOLO throws out the pipeline entirely and is fast by design.

Detectors for single classes like faces or people can be highly optimized since they have to deal with much less variation [37]. YOLO is a general purpose detector that learns to detect a variety of objects simultaneously.

Deep MultiBox. Unlike R-CNN, Szegedy et al. train a convolutional neural network to predict regions of interest [8] instead of using Selective Search. MultiBox can also perform single object detection by replacing the confidence prediction with a single class prediction. However, MultiBox cannot perform general object detection and is still just a piece in a larger detection pipeline, requiring further image patch classification. Both YOLO and MultiBox use a convolutional network to predict bounding boxes in an image but YOLO is a complete detection system.

OverFeat. Sermanet et al. train a convolutional neural network to perform localization and adapt that localizer to perform detection [32]. OverFeat efficiently performs sliding window detection but it is still a disjoint system. OverFeat optimizes for localization, not detection performance. Like DPM, the localizer only sees local information when making a prediction. OverFeat cannot reason about global context and thus requires significant post-processing to produce coherent detections.

MultiGrasp. Our work is similar in design to work on

grasp detection by Redmon et al [27]. Our grid approach to bounding box prediction is based on the MultiGrasp system for regression to grasps. However, grasp detection is a much simpler task than object detection. MultiGrasp only needs to predict a single graspable region for an image containing one object. It doesn't have to estimate the size, location, or boundaries of the object or predict it's class, only find a region suitable for grasping. YOLO predicts both bounding boxes and class probabilities for multiple objects of multiple classes in an image.

4. Experiments

First we compare YOLO with other real-time detection systems on PASCAL VOC 2007. To understand the differences between YOLO and R-CNN variants we explore the errors on VOC 2007 made by YOLO and Fast R-CNN, one of the highest performing versions of R-CNN [14]. Based on the different error profiles we show that YOLO can be used to rescore Fast R-CNN detections and reduce the errors from background false positives, giving a significant performance boost. We also present VOC 2012 results and compare mAP to current state-of-the-art methods. Finally, we show that YOLO generalizes to new domains better than other detectors on two artwork datasets.

4.1. Comparison to Other Real-Time Systems

Many research efforts in object detection focus on making standard detection pipelines fast. [5] [38] [31] [14] [17] [28] However, only Sadeghi et al. actually produce a detection system that runs in real-time (30 frames per second or better) [31]. We compare YOLO to their GPU implementation of DPM which runs either at 30Hz or 100Hz. While the other efforts don't reach the real-time milestone we also compare their relative mAP and speed to examine the accuracy-performance tradeoffs available in object detection systems.

Fast YOLO is the fastest object detection method on PASCAL; as far as we know, it is the fastest extant object detector. With 52.7% mAP, it is more than twice as accurate as prior work on real-time detection. YOLO pushes mAP to 63.4% while still maintaining real-time performance.

We also train YOLO using VGG-16. This model is more accurate but also significantly slower than YOLO. It is useful for comparison to other detection systems that rely on VGG-16 but since it is slower than real-time the rest of the paper focuses on our faster models.

Fastest DPM effectively speeds up DPM without sacrificing much mAP but it still misses real-time performance by a factor of 2 [38]. It also is limited by DPM's relatively low accuracy on detection compared to neural network approaches.

R-CNN minus R replaces Selective Search with static bounding box proposals [20]. While it is much faster than

搜索[35]生成潜在的边界框，卷积网络提取特征，SVM对边界框进行评分，线性模型调整边界框，非极大值抑制消除重复检测。这个复杂流程的每个阶段都必须独立进行精确调优，且最终系统非常缓慢，在测试时每张图像的处理时间超过40秒[14]。

YOLO与R-CNN存在某些相似之处。每个网格单元会提出潜在的边界框，并利用卷积特征对这些框进行评分。然而，我们的系统对网格单元的提议施加了空间约束，这有助于减少对同一物体的多次检测。我们的系统提出的边界框数量也少得多，每张图像仅98个，而选择性搜索方法约为2000个。最后，我们的系统将这些独立组件整合为一个统一、联合优化的模型。

其他快速检测器 Fast R-CNN 和 Faster R-CNN 通过共享计算以及使用神经网络代替选择性搜索来提出区域，从而专注于加速 R-CNN 框架 [14][28]。尽管它们在速度和准确率上相比 R-CNN 有所提升，但两者仍未能实现实时性能。

许多研究工作致力于加速DPM流水线[31][38][5]。它们通过加速HOG计算、采用级联结构以及将计算任务转移至GPU来实现加速。然而，只有30Hz DPM[31]真正实现了实时运行。

YOLO摒弃了传统大型检测流程中逐个优化组件的思路，彻底重构了整体架构，其速度优势源于根本性的设计革新。

针对单一类别（如人脸或行人）的检测器可以实现高度优化，因为它们需要处理的变异性要小得多[37]。YOLO是一种通用检测器，它能同时学习检测多种目标。

深度MultiBox。与R-CNN不同，Szegedy等人训练了一个卷积神经网络来预测感兴趣区域[8]，而不是使用选择性搜索。MultiBox也可以通过将置信度预测替换为单一类别预测来执行单目标检测。然而，MultiBox无法执行通用目标检测，并且仍然只是更大检测流程中的一个环节，需要进一步的图像块分类。YOLO和MultiBox都使用卷积网络来预测图像中的边界框，但YOLO是一个完整的检测系统。

OverFeat。Sermanet等人训练了一个卷积神经网络进行定位，并将该定位器调整用于检测[32]。OverFeat高效地执行滑动窗口检测，但它仍然是一个分离的系统。OverFeat针对定位而非检测性能进行了优化。与DPM类似，定位器在做出预测时仅能看到局部信息。OverFeat无法推理全局上下文，因此需要大量的后处理来生成连贯的检测结果。

MultiGrasp。我们的工作在设计上类似于对

Redmon等人[27]提出的抓取检测方法。我们采用网格化方法进行边界框预测，其基础是用于抓取回归的MultiGrasp系统。然而，抓取检测任务远比目标检测简单。MultiGrasp仅需为包含单个物体的图像预测一个可抓取区域，无需估计物体尺寸、位置或边界，也无需预测类别，只需找到适合抓取的区域。而YOLO则能同时预测图像中多个类别物体的边界框和类别概率。

4. 实验

首先，我们在PASCAL VOC 2007数据集上将YOLO与其他实时检测系统进行比较。为了理解YOLO与R-CNN变体之间的差异，我们分析了YOLO和Fast R-CNN（R-CNN系列中性能最高的版本之一[14]）在VOC 2007上的错误类型。基于不同的错误特征，我们证明YOLO可用于重新评分Fast R-CNN的检测结果，减少由背景误报引起的错误，从而显著提升性能。我们还展示了VOC 2012的结果，并将mAP与当前最先进的方法进行比较。最后，我们通过两个艺术品数据集证明，YOLO在新领域中的泛化能力优于其他检测器。

4.1. 与其他实时系统的比较

物体检测领域的许多研究工作都致力于提升标准检测流程的速度。[5][38][31][14][17][28] 然而，只有Sadeghi等人真正实现了实时运行的检测系统（每秒30帧或更高）[31]。我们将YOLO与其GPU实现的DPM进行对比，后者的运行频率为30Hz或100Hz。尽管其他研究未能达到实时性标准，我们仍通过对比其相对mAP和速度，以探究物体检测系统中精度与性能的权衡关系。

Fast YOLO 是 PASCAL 上最快的物体检测方法；据我们所知，它也是现有最快的物体检测器。凭借 52.7 % 的 mAP，其准确率比以往的实时检测方法提高了一倍以上。YOLO 将 mAP 提升至 63.4%，同时仍保持实时性能。

我们还使用VGG-16训练了YOLO模型。该模型比YOLO更精确，但也明显更慢。虽然它有助于与其他依赖VGG-16的检测系统进行比较，但由于其速度低于实时要求，本文后续将主要聚焦于我们更快的模型。

最快的DPM在不牺牲太多mAP的情况下有效加速了DPM，但其实时性能仍有两倍差距[38]。同时，与神经网络方法相比，DPM在检测精度上的相对不足也限制了其发展。

R-CNN减去R用静态边界框建议取代了选择性搜索[20]。虽然它比

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 1: Real-Time Systems on PASCAL VOC 2007. Comparing the performance and speed of fast detectors. Fast YOLO is the fastest detector on record for PASCAL VOC detection and is still twice as accurate as any other real-time detector. YOLO is 10 mAP more accurate than the fast version while still well above real-time in speed.

R-CNN, it still falls short of real-time and takes a significant accuracy hit from not having good proposals.

Fast R-CNN speeds up the classification stage of R-CNN but it still relies on selective search which can take around 2 seconds per image to generate bounding box proposals. Thus it has high mAP but at 0.5 fps it is still far from real-time.

The recent Faster R-CNN replaces selective search with a neural network to propose bounding boxes, similar to Szegedy et al. [8] In our tests, their most accurate model achieves 7 fps while a smaller, less accurate one runs at 18 fps. The VGG-16 version of Faster R-CNN is 10 mAP higher but is also 6 times slower than YOLO. The Zeiler-Fergus Faster R-CNN is only 2.5 times slower than YOLO but is also less accurate.

4.2. VOC 2007 Error Analysis

To further examine the differences between YOLO and state-of-the-art detectors, we look at a detailed breakdown of results on VOC 2007. We compare YOLO to Fast R-CNN since Fast R-CNN is one of the highest performing detectors on PASCAL and it's detections are publicly available.

We use the methodology and tools of Hoiem et al. [19] For each category at test time we look at the top N predictions for that category. Each prediction is either correct or it is classified based on the type of error:

- Correct: correct class and $\text{IOU} > .5$
- Localization: correct class, $.1 < \text{IOU} < .5$
- Similar: class is similar, $\text{IOU} > .1$

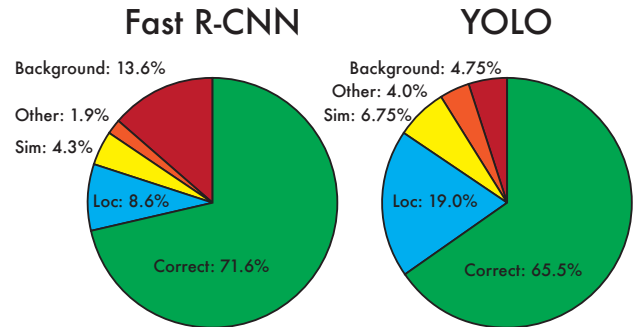


Figure 4: Error Analysis: Fast R-CNN vs. YOLO These charts show the percentage of localization and background errors in the top N detections for various categories ($N = \#$ objects in that category).

- Other: class is wrong, $\text{IOU} > .1$
- Background: $\text{IOU} < .1$ for any object

Figure 4 shows the breakdown of each error type averaged across all 20 classes.

YOLO struggles to localize objects correctly. Localization errors account for more of YOLO's errors than all other sources combined. Fast R-CNN makes much fewer localization errors but far more background errors. 13.6% of it's top detections are false positives that don't contain any objects. Fast R-CNN is almost 3x more likely to predict background detections than YOLO.

4.3. Combining Fast R-CNN and YOLO

YOLO makes far fewer background mistakes than Fast R-CNN. By using YOLO to eliminate background detections from Fast R-CNN we get a significant boost in performance. For every bounding box that R-CNN predicts we check to see if YOLO predicts a similar box. If it does, we give that prediction a boost based on the probability predicted by YOLO and the overlap between the two boxes.

The best Fast R-CNN model achieves a mAP of 71.8% on the VOC 2007 test set. When combined with YOLO, its

	mAP	Combined	Gain
Fast R-CNN	71.8	-	-
Fast R-CNN (2007 data)	66.9	72.4	.6
Fast R-CNN (VGG-M)	59.2	72.4	.6
Fast R-CNN (CaffeNet)	57.1	72.1	.3
YOLO	63.4	75.0	3.2

Table 2: Model combination experiments on VOC 2007. We examine the effect of combining various models with the best version of Fast R-CNN. Other versions of Fast R-CNN provide only a small benefit while YOLO provides a significant performance boost.

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

表1: PASCAL VOC 2007上的实时系统。比较快速检测器的性能与速度。Fast YOLO是PASCAL VOC检测中记录最快的检测器，其准确率仍是其他实时检测器的两倍。YOLO比快速版本的准确率高出10 mAP，同时速度仍远高于实时要求。

R-CNN仍然无法实现实时处理，并且由于缺乏高质量的候选区域，其准确性受到了显著影响。

Fast R-CNN 加速了 R-CNN 的分类阶段，但它仍然依赖于选择性搜索，每张图像生成边界框建议大约需要 2 秒。因此，它具有较高的 mAP，但在 0.5 fps 的帧率下，仍远未达到实时性能。

最近的Faster R-CNN用神经网络取代了选择性搜索来提出边界框，类似于Szegedy等人[8]的方法。在我们的测试中，他们最精确的模型达到每秒7帧，而一个更小、精度较低的模型则以每秒18帧运行。Faster R-CNN的VGG-16版本比YOLO的mAP高出10点，但速度也慢了6倍。Zeiler-Fergus版本的Faster R-CNN仅比YOLO慢2.5倍，但精度也较低。

4.2. VOC 2007 错误分析

为了进一步检验YOLO与先进检测器之间的差异，我们详细分析了VOC 2007的结果细分。我们将YOLO与Fast R-CNN进行对比，因为Fast R-CNN是PASCAL数据集上性能最优的检测器之一，且其检测结果已公开可用。

我们采用Hoiem等人[19]的方法论与工具。在测试阶段，针对每个类别，我们查看该类别的前N个预测结果。每个预测要么正确，要么根据错误类型进行分类：

- 正确：正确类别和 $\text{IOU} > .5$
- 本地化：正确类别， $.1 < \text{交并比} < .5$
- 相似：类别相似， $\text{IOU} > .1$

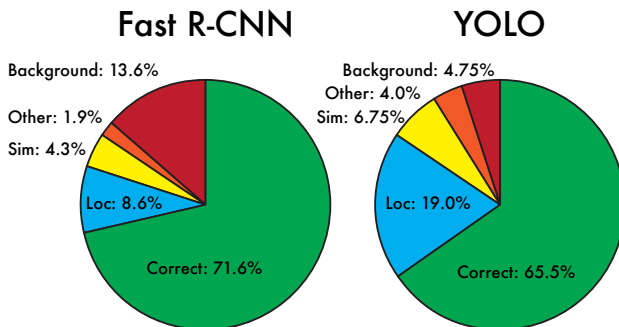


图4: 错误分析: Fast R-CNN 与 YOLO 对比 这些图表展示了各类别 (N = 为该类别中的目标数量) 在前 N 个检测结果中定位错误与背景错误的百分比。

- 其他：类别错误， $\text{IOU} > .1$
- 背景：任何物体的 $\text{IOU} \{v^*\}1$

图4展示了所有20个类别中各类错误的平均细分情况。

YOLO在准确定位物体方面存在困难。其定位误差超过了所有其他误差来源的总和。Fast R-CNN的定位误差要少得多，但背景误判却多得多。其最高置信度检测结果中有13.6%是不包含任何物体的误报。Fast R-CNN预测背景检测的概率几乎是YOLO的3倍。

4.3. 结合Fast R-CNN与YOLO

YOLO比Fast R-CNN的背景误判少得多。通过使用YOLO消除Fast R-CNN的背景检测，我们获得了显著的性能提升。对于R-CNN预测的每个边界框，我们会检查YOLO是否预测了相似的框。如果存在，则根据YOLO预测的概率以及两个框之间的重叠程度，对该预测进行加权增强。

最佳的Fast R-CNN模型在VOC 2007测试集上实现了71.8%的mAP。当与YOLO结合时，其

	mAP	Combined	Gain
Fast R-CNN	71.8	-	-
Fast R-CNN (2007 data)	66.9	72.4	.6
Fast R-CNN (VGG-M)	59.2	72.4	.6
Fast R-CNN (CaffeNet)	57.1	72.1	.3
YOLO	63.4	75.0	3.2

表2: 在VOC 2007上进行的模型组合实验。我们研究了将不同模型与Fast R-CNN的最佳版本相结合的效果。Fast R-CNN的其他版本仅带来小幅性能提升，而YOLO则显著提高了性能。

VOC 2012 test	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
MR_CNN_MORE.DATA [11]	73.9	85.5	82.9	76.6	57.8	62.7	79.4	77.2	86.6	55.0	79.1	62.2	87.0	83.4	84.7	78.9	45.3	73.4	65.8	80.3	74.0
HyperNet_VGG	71.4	84.2	78.5	73.6	55.6	53.7	78.7	79.8	87.7	49.6	74.9	52.1	86.0	81.7	83.3	81.8	48.6	73.5	59.4	79.9	65.7
HyperNet_SP	71.3	84.1	78.3	73.3	55.5	53.6	78.6	79.6	87.5	49.5	74.9	52.1	85.6	81.6	83.2	81.6	48.4	73.2	59.3	79.7	65.6
Fast R-CNN + YOLO	70.7	83.4	78.5	73.5	55.8	43.4	79.1	73.1	89.4	49.4	75.5	57.0	87.5	80.9	81.0	74.7	41.8	71.5	68.5	82.1	67.2
MR_CNN_S_CNN [11]	70.7	85.0	79.6	71.5	55.3	57.7	76.0	73.9	84.6	50.5	74.3	61.7	85.5	79.9	81.7	76.4	41.0	69.0	61.2	77.7	72.1
Faster R-CNN [28]	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
DEEP_ENS_COCO	70.1	84.0	79.4	71.6	51.9	51.1	74.1	72.1	88.6	48.3	73.4	57.8	86.1	80.0	80.7	70.4	46.6	69.6	68.8	75.9	71.4
NoC [29]	68.8	82.8	79.0	71.6	52.3	53.7	74.1	69.0	84.9	46.9	74.3	53.1	85.0	81.3	79.5	72.2	38.9	72.4	59.5	76.7	68.1
Fast R-CNN [14]	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	87.5	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
UMICH_FGS_STRUCT	66.4	82.9	76.1	64.1	44.6	49.4	70.3	71.2	84.6	42.7	68.6	55.8	82.7	77.1	79.9	68.7	41.4	69.0	60.0	72.0	66.2
NUS_NIN_C2000 [7]	63.8	80.2	73.8	61.9	43.7	43.0	70.3	67.6	80.7	41.9	69.7	51.7	78.2	75.2	76.9	65.1	38.6	68.3	58.0	68.7	63.3
BabyLearning [7]	63.2	78.0	74.2	61.3	45.7	42.7	68.2	66.8	80.2	40.6	70.0	49.8	79.0	74.5	77.9	64.0	35.3	67.9	55.7	68.7	62.6
NUS_NIN	62.4	77.9	73.1	62.6	39.5	43.3	69.1	66.4	78.9	39.1	68.1	50.0	77.2	71.3	76.1	64.7	38.4	66.9	56.2	66.9	62.7
R-CNN VGG BB [13]	62.4	79.6	72.7	61.9	41.2	41.9	65.9	66.4	84.6	38.5	67.2	46.7	82.0	74.8	76.0	65.2	35.6	65.4	54.2	67.4	60.3
R-CNN VGG [13]	59.2	76.8	70.9	56.6	37.5	36.9	62.9	63.6	81.1	35.7	64.3	43.9	80.4	71.6	74.0	60.0	30.8	63.4	52.0	63.5	58.7
YOLO	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
Feature Edit [33]	56.3	74.6	69.1	54.4	39.1	33.1	65.2	62.7	69.7	30.8	56.0	44.6	70.0	64.4	71.1	60.2	33.3	61.3	46.4	61.7	57.8
R-CNN BB [13]	53.3	71.8	65.8	52.0	34.1	32.6	59.6	60.0	69.8	27.6	52.0	41.7	69.6	61.3	68.3	57.8	29.6	57.8	40.9	59.3	54.1
SDS [16]	50.7	69.7	58.4	48.5	28.3	28.8	61.3	57.5	70.8	24.1	50.7	35.9	64.9	59.1	65.8	57.1	26.0	58.8	38.6	58.9	50.7
R-CNN [13]	49.6	68.1	63.8	46.1	29.4	27.9	56.6	57.0	65.9	26.5	48.7	39.5	66.2	57.3	65.4	53.2	26.2	54.5	38.1	50.6	51.6

Table 3: PASCAL VOC 2012 Leaderboard. YOLO compared with the full comp4 (outside data allowed) public leaderboard as of November 6th, 2015. Mean average precision and per-class average precision are shown for a variety of detection methods. YOLO is the only real-time detector. Fast R-CNN + YOLO is the forth highest scoring method, with a 2.3% boost over Fast R-CNN.

mAP increases by 3.2% to 75.0%. We also tried combining the top Fast R-CNN model with several other versions of Fast R-CNN. Those ensembles produced small increases in mAP between .3 and .6%, see Table 2 for details.

The boost from YOLO is not simply a byproduct of model ensembling since there is little benefit from combining different versions of Fast R-CNN. Rather, it is precisely because YOLO makes different kinds of mistakes at test time that it is so effective at boosting Fast R-CNN’s performance.

Unfortunately, this combination doesn’t benefit from the speed of YOLO since we run each model separately and then combine the results. However, since YOLO is so fast it doesn’t add any significant computational time compared to Fast R-CNN.

4.4. VOC 2012 Results

On the VOC 2012 test set, YOLO scores 57.9% mAP. This is lower than the current state of the art, closer to the original R-CNN using VGG-16, see Table 3. Our system struggles with small objects compared to its closest competitors. On categories like *bottle*, *sheep*, and *tv/monitor* YOLO scores 8-10% lower than R-CNN or Feature Edit. However, on other categories like *cat* and *train* YOLO achieves higher performance.

Our combined Fast R-CNN + YOLO model is one of the highest performing detection methods. Fast R-CNN gets a 2.3% improvement from the combination with YOLO, boosting it 5 spots up on the public leaderboard.

4.5. Generalizability: Person Detection in Artwork

Academic datasets for object detection draw the training and testing data from the same distribution. In real-world applications it is hard to predict all possible use cases and

the test data can diverge from what the system has seen before [3]. We compare YOLO to other detection systems on the Picasso Dataset [12] and the People-Art Dataset [3], two datasets for testing person detection on artwork.

Figure 5 shows comparative performance between YOLO and other detection methods. For reference, we give VOC 2007 detection AP on *person* where all models are trained only on VOC 2007 data. On Picasso models are trained on VOC 2012 while on People-Art they are trained on VOC 2010.

R-CNN has high AP on VOC 2007. However, R-CNN drops off considerably when applied to artwork. R-CNN uses Selective Search for bounding box proposals which is tuned for natural images. The classifier step in R-CNN only sees small regions and needs good proposals.

DPM maintains its AP well when applied to artwork. Prior work theorizes that DPM performs well because it has strong spatial models of the shape and layout of objects. Though DPM doesn’t degrade as much as R-CNN, it starts from a lower AP.

YOLO has good performance on VOC 2007 and its AP degrades less than other methods when applied to artwork. Like DPM, YOLO models the size and shape of objects, as well as relationships between objects and where objects commonly appear. Artwork and natural images are very different on a pixel level but they are similar in terms of the size and shape of objects, thus YOLO can still predict good bounding boxes and detections.

5. Real-Time Detection In The Wild

YOLO is a fast, accurate object detector, making it ideal for computer vision applications. We connect YOLO to a webcam and verify that it maintains real-time performance,

VOC 2012 test	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
MR_CNN_MORE.DATA [11]	73.9	85.5	82.9	76.6	57.8	62.7	79.4	77.2	86.6	55.0	79.1	62.2	87.0	83.4	84.7	78.9	45.3	73.4	65.8	80.3	74.0
HyperNet_VGG	71.4	84.2	78.5	73.6	55.6	53.7	78.7	79.8	87.7	49.6	74.9	52.1	86.0	81.7	83.3	81.8	48.6	73.5	59.4	79.9	65.7
HyperNet_SP	71.3	84.1	78.3	73.3	55.5	53.6	78.6	79.6	87.5	49.5	74.9	52.1	85.6	81.6	83.2	81.6	48.4	73.2	59.3	79.7	65.6
Fast R-CNN + YOLO	70.7	83.4	78.5	73.5	55.8	43.4	79.1	73.1	89.4	49.4	75.5	57.0	87.5	80.9	81.0	74.7	41.8	71.5	68.5	82.1	67.2
MR_CNN_S_CNN [11]	70.7	85.0	79.6	71.5	55.3	57.7	76.0	73.9	84.6	50.5	74.3	61.7	85.5	79.9	81.7	76.4	41.0	69.0	61.2	77.7	72.1
Faster R-CNN [28]	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
DEEP_ENS_COCO	70.1	84.0	79.4	71.6	51.9	51.1	74.1	72.1	88.6	48.3	73.4	57.8	86.1	80.0	80.7	70.4	46.6	69.6	68.8	75.9	71.4
NoC [29]	68.8	82.8	79.0	71.6	52.3	53.7	74.1	69.0	84.9	46.9	74.3	53.1	85.0	81.3	79.5	72.2	38.9	72.4	59.5	76.7	68.1
Fast R-CNN [14]	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	87.5	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
UMICH_FGS_STRUCT	66.4	82.9	76.1	64.1	44.6	49.4	70.3	71.2	84.6	42.7	68.6	55.8	82.7	77.1	79.9	68.7	41.4	69.0	60.0	72.0	66.2
NUS_NIN_C2000 [7]	63.8	80.2	73.8	61.9	43.7	43.0	70.3	67.6	80.7	41.9	69.7	51.7	78.2	75.2	76.9	65.1	38.6	68.3	58.0	68.7	63.3
BabyLearning [7]	63.2	78.0	74.2	61.3	45.7	42.7	68.2	66.8	80.2	40.6	70.0	49.8	79.0	74.5	77.9	64.0	35.3	67.9	55.7	68.7	62.6
NUS_NIN	62.4	77.9	73.1	62.6	39.5	43.3	69.1	66.4	78.9	39.1	68.1	50.0	77.2	71.3	76.1	64.7	38.4	66.9	56.2	66.9	62.7
R-CNN VGG BB [13]	62.4	79.6	72.7	61.9	41.2	41.9	65.9	66.4	84.6	38.5	67.2	46.7	82.0	74.8	76.0	65.2	35.6	65.4	54.2	67.4	60.3
R-CNN VGG [13]	59.2	76.8	70.9	56.6	37.5	36.9	62.9	63.6	81.1	35.7	64.3	43.9	80.4	71.6	74.0	60.0	30.8	63.4	52.0	63.5	58.7
YOLO	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
Feature Edit [33]	56.3	74.6	69.1	54.4	39.1	33.1	65.2	62.7	69.7	30.8	56.0	44.6	70.0	64.4	71.1	60.2	33.3	61.3	46.4	61.7	57.8
R-CNN BB [13]	53.3	71.8	65.8	52.0	34.1	32.6	59.6	60.0	69.8	27.6	52.0	41.7	69.6	61.3	68.3	57.8	29.6	57.8	40.9	59.3	54.1
SDS [16]	50.7	69.7	58.4	48.5	28.3	28.8	61.3	57.5	70.8	24.1	50.7	35.9	64.9	59.1	65.8	57.1	26.0	58.8	38.6	58.9	50.7
R-CNN [13]	49.6	68.1	63.8	46.1	29.4	27.9	56.6	57.0	65.9	26.5	48.7	39.5	66.2	57.3	65.4	53.2	26.2	54.5	38.1	50.6	51.6

表3: PASCAL VOC 2012排行榜。YOLO与截至2015年11月6日完整的comp4（允许外部数据）公开排行榜对比。表中展示了多种检测方法的平均精度均值及各类平均精度。YOLO是唯一的实时检测器。Fast R-CNN + YOLO位列第四高分方法，较Fast R-CNN提升2.3%。

mAP提升了3.2%，达到75.0%。我们还尝试将表现最佳的Fast R-CNN模型与其他几个版本的Fast R-CNN相结合。这些集成模型使mAP小幅提升了0.3%至0.6%，详见表2。

YOLO带来的性能提升并非仅仅是模型集成的副产品，因为结合不同版本的Fast R-CNN收效甚微。恰恰是由于YOLO在测试时会产生不同类型的错误，它才能如此有效地提升Fast R-CNN的性能。

遗憾的是，这种组合并未受益于YOLO的速度优势，因为我们是分别运行每个模型后再合并结果。不过，由于YOLO本身速度极快，相比Fast R-CNN而言，它并未增加任何显著的计算时间。

4.4. VOC 2012 结果

在VOC 2012测试集上，YOLO获得了57.9%的mAP分数。这低于当前的最先进水平，更接近使用VGG-16的原始R-CNN，详见表3。与最接近的竞争对手相比，我们的系统在处理小物体时表现欠佳。在瓶子、绵羊和电视/显示器等类别上，YOLO的得分比R-CNN或特征编辑低8-10%。然而，在猫和火车等其他类别上，YOLO实现了更高的性能。

我们结合Fast R-CNN +与YOLO的模型是性能最高的检测方法之一。通过与YOLO结合，Fast R-CNN获得了2.3%的性能提升，使其在公开排行榜上跃升了5个名次。

4.5. 泛化性：艺术品中的人物检测

用于目标检测的学术数据集从同一分布中抽取训练和测试数据。在实际应用中，很难预测所有可能的使用场景，并且

测试数据可能与系统之前见过的数据有所不同[3]。我们在毕加索数据集[12]和People-Art数据集[3]上将YOLO与其他检测系统进行比较，这两个数据集用于测试艺术品中的人物检测。

图5展示了YOLO与其他检测方法的性能比较。作为参考，我们提供了在VOC 2007数据集上仅使用VOC 2007数据训练的所有模型在“人物”类别上的检测AP值。对于Picasso模型，其训练数据为VOC 2012；而对于People-Art模型，则使用VOC 2010数据进行训练。

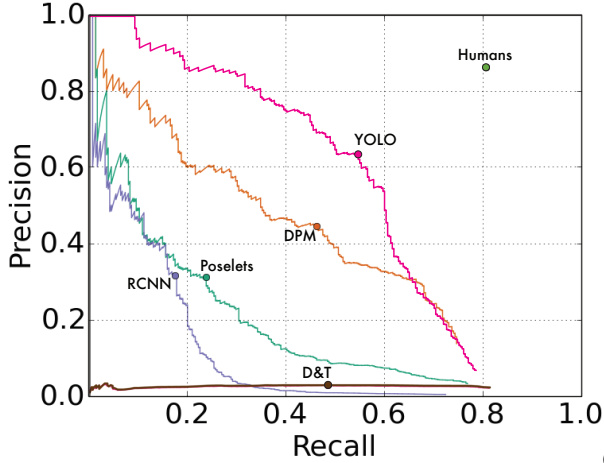
R-CNN在VOC 2007数据集上具有较高的平均精度（AP）。然而，当应用于艺术作品时，R-CNN的性能显著下降。R-CNN使用针对自然图像优化的选择性搜索（Selective Search）来生成边界框建议。R-CNN的分类器步骤仅关注小区域，因此需要高质量的建议框。

DPM在应用于艺术作品时能较好地保持其AP。先前的研究理论认为，DPM表现出色是因为它对物体的形状和布局有强大的空间建模能力。尽管DPM的性能下降不如R-CNN明显，但其初始AP较低。

YOLO在VOC 2007数据集上表现出色，当应用于艺术作品时其AP值下降幅度小于其他方法。与DPM类似，YOLO能够建模物体的尺寸、形状、物体间关系以及物体常见出现位置。虽然艺术作品与自然图像在像素层面差异显著，但它们在物体尺寸和形状方面具有相似性，因此YOLO仍能预测出优质的边界框和检测结果。

5. 野外实时检测

YOLO是一种快速、准确的目标检测器，使其成为计算机视觉应用的理想选择。我们将YOLO连接到网络摄像头，并验证其保持了实时性能。



(a) Picasso Dataset precision-recall curves.

	VOC 2007 AP	Picasso AP Best F_1	People-Art AP
YOLO	59.2	53.3 0.590	45
R-CNN	54.2	10.4 0.226	26
DPM	43.2	37.8 0.458	32
Poselets [2]	36.5	17.8 0.271	
D&T [4]	-	1.9 0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Figure 5: Generalization results on Picasso and People-Art datasets.



Figure 6: Qualitative Results. YOLO running on sample artwork and natural images from the internet. It is mostly accurate although it does think one person is an airplane.

including the time to fetch images from the camera and display the detections.

The resulting system is interactive and engaging. While YOLO processes images individually, when attached to a webcam it functions like a tracking system, detecting objects as they move around and change in appearance. A demo of the system and the source code can be found on our project website: <http://pjreddie.com/yolo/>.

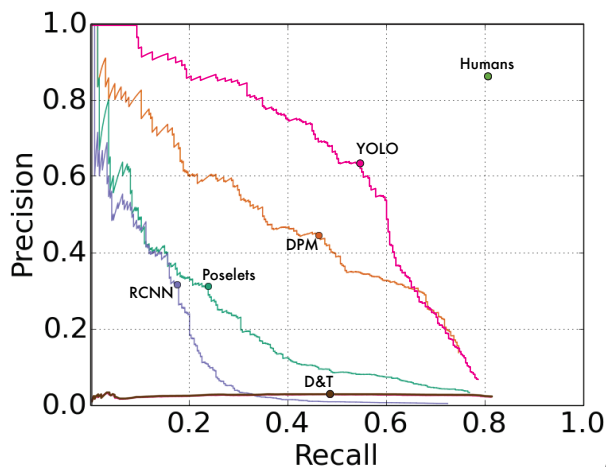
6. Conclusion

We introduce YOLO, a unified model for object detection. Our model is simple to construct and can be trained

directly on full images. Unlike classifier-based approaches, YOLO is trained on a loss function that directly corresponds to detection performance and the entire model is trained jointly.

Fast YOLO is the fastest general-purpose object detector in the literature and YOLO pushes the state-of-the-art in real-time object detection. YOLO also generalizes well to new domains making it ideal for applications that rely on fast, robust object detection.

Acknowledgements: This work is partially supported by ONR N00014-13-1-0720, NSF IIS-1338054, and The Allen Distinguished Investigator Award.



(a) Picasso Dataset precision-recall curves.

	VOC 2007 AP	Picasso AP Best F_1	People-Art AP
YOLO	59.2	53.3 0.590	45
R-CNN	54.2	10.4 0.226	26
DPM	43.2	37.8 0.458	32
Poselets [2]	36.5	17.8 0.271	
D&T [4]	-	1.9 0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

图5：毕加索和People-Art数据集上的泛化结果。

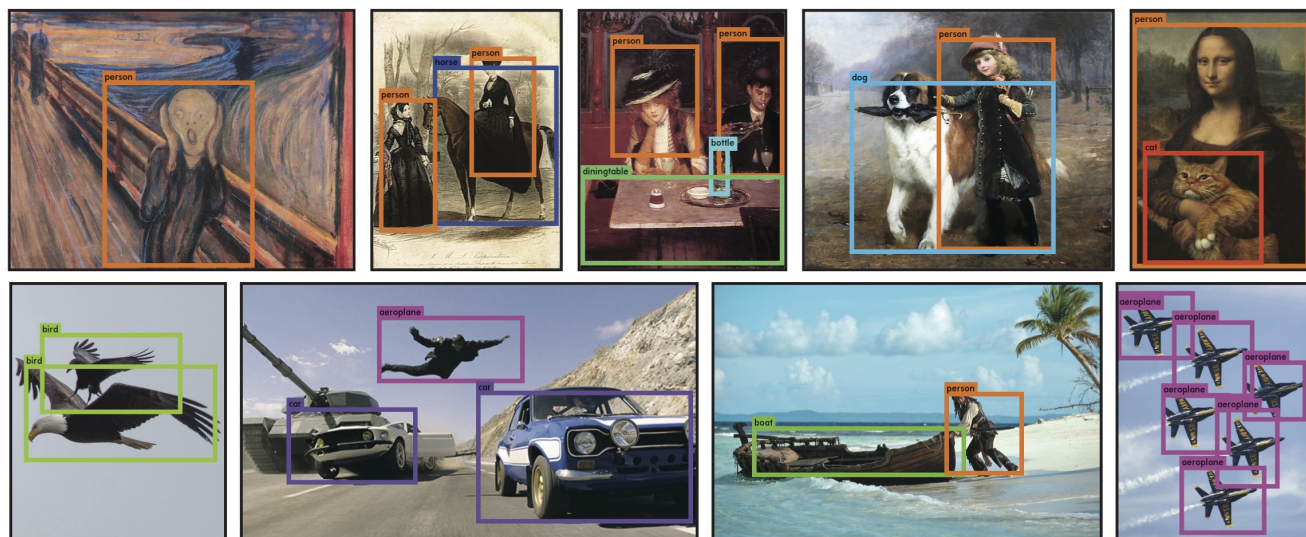


图6：定性结果。YOLO在来自互联网的样本艺术作品和自然图像上运行。虽然它误将一个人识别为飞机，但总体准确率较高。

包括从相机获取图像和显示检测结果的时间。

最终形成的系统具有交互性和吸引力。虽然YOLO单独处理图像，但当连接到网络摄像头时，它就像一个跟踪系统，能够检测移动和外观变化的物体。该系统的演示和源代码可在我们的项目网站上找到：<http://pjreddie.com/yolo/>。

6. 结论

我们介绍YOLO，一个用于目标检测的统一模型。我们的模型结构简单，易于训练

直接在全图像上进行。与基于分类器的方法不同，YOLO通过直接对应检测性能的损失函数进行训练，且整个模型是联合训练的。

Fast YOLO是文献中最快的通用目标检测器，而YOLO则将实时目标检测的技术水平推向了前沿。YOLO还能很好地泛化到新领域，使其成为依赖快速、鲁棒目标检测应用的理想选择。

致谢：本工作部分得到了ONR N00014-13-1-0720、NSF IIS-1338054以及艾伦杰出研究员奖的支持。

References

- [1] M. B. Blaschko and C. H. Lampert. Learning to localize objects with structured output regression. In *Computer Vision–ECCV 2008*, pages 2–15. Springer, 2008. 4
- [2] L. Bourdev and J. Malik. Poselets: Body part detectors trained using 3d human pose annotations. In *International Conference on Computer Vision (ICCV)*, 2009. 8
- [3] H. Cai, Q. Wu, T. Corradi, and P. Hall. The cross-depiction problem: Computer vision algorithms for recognising objects in artwork and in photographs. *arXiv preprint arXiv:1505.00110*, 2015. 7
- [4] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In *Computer Vision and Pattern Recognition, 2005. CVPR 2005. IEEE Computer Society Conference on*, volume 1, pages 886–893. IEEE, 2005. 4, 8
- [5] T. Dean, M. Ruzon, M. Segal, J. Shlens, S. Vijayanarasimhan, J. Yagnik, et al. Fast, accurate detection of 100,000 object classes on a single machine. In *Computer Vision and Pattern Recognition (CVPR), 2013 IEEE Conference on*, pages 1814–1821. IEEE, 2013. 5
- [6] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell. Decaf: A deep convolutional activation feature for generic visual recognition. *arXiv preprint arXiv:1310.1531*, 2013. 4
- [7] J. Dong, Q. Chen, S. Yan, and A. Yuille. Towards unified object detection and semantic segmentation. In *Computer Vision–ECCV 2014*, pages 299–314. Springer, 2014. 7
- [8] D. Erhan, C. Szegedy, A. Toshev, and D. Anguelov. Scalable object detection using deep neural networks. In *Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on*, pages 2155–2162. IEEE, 2014. 5, 6
- [9] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The pascal visual object classes challenge: A retrospective. *International Journal of Computer Vision*, 111(1):98–136, Jan. 2015. 2
- [10] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan. Object detection with discriminatively trained part based models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(9):1627–1645, 2010. 1, 4
- [11] S. Gidaris and N. Komodakis. Object detection via a multi-region & semantic segmentation-aware CNN model. *CoRR*, abs/1505.01749, 2015. 7
- [12] S. Ginosar, D. Haas, T. Brown, and J. Malik. Detecting people in cubist art. In *Computer Vision–ECCV 2014 Workshops*, pages 101–116. Springer, 2014. 7
- [13] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on*, pages 580–587. IEEE, 2014. 1, 4, 7
- [14] R. B. Girshick. Fast R-CNN. *CoRR*, abs/1504.08083, 2015. 2, 5, 6, 7
- [15] S. Gould, T. Gao, and D. Koller. Region-based segmentation and object detection. In *Advances in neural information processing systems*, pages 655–663, 2009. 4
- [16] B. Hariharan, P. Arbeláez, R. Girshick, and J. Malik. Simultaneous detection and segmentation. In *Computer Vision–ECCV 2014*, pages 297–312. Springer, 2014. 7
- [17] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. *arXiv preprint arXiv:1406.4729*, 2014. 5
- [18] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012. 4
- [19] D. Hoiem, Y. Chodpathumwan, and Q. Dai. Diagnosing error in object detectors. In *Computer Vision–ECCV 2012*, pages 340–353. Springer, 2012. 6
- [20] K. Lenc and A. Vedaldi. R-cnn minus r. *arXiv preprint arXiv:1506.06981*, 2015. 5, 6
- [21] R. Lienhart and J. Maydt. An extended set of haar-like features for rapid object detection. In *Image Processing. 2002. Proceedings. 2002 International Conference on*, volume 1, pages I–900. IEEE, 2002. 4
- [22] M. Lin, Q. Chen, and S. Yan. Network in network. *CoRR*, abs/1312.4400, 2013. 2
- [23] D. G. Lowe. Object recognition from local scale-invariant features. In *Computer vision, 1999. The proceedings of the seventh IEEE international conference on*, volume 2, pages 1150–1157. Ieee, 1999. 4
- [24] D. Mishkin. Models accuracy on imagenet 2012 val. <https://github.com/BVLC/caffe/wiki/Models-accuracy-on-ImageNet-2012-val>. Accessed: 2015-10-2. 3
- [25] C. P. Papageorgiou, M. Oren, and T. Poggio. A general framework for object detection. In *Computer vision, 1998. sixth international conference on*, pages 555–562. IEEE, 1998. 4
- [26] J. Redmon. Darknet: Open source neural networks in c. <http://pjreddie.com/darknet/>, 2013–2016. 3
- [27] J. Redmon and A. Angelova. Real-time grasp detection using convolutional neural networks. *CoRR*, abs/1412.3128, 2014. 5
- [28] S. Ren, K. He, R. Girshick, and J. Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *arXiv preprint arXiv:1506.01497*, 2015. 5, 6, 7
- [29] S. Ren, K. He, R. B. Girshick, X. Zhang, and J. Sun. Object detection networks on convolutional feature maps. *CoRR*, abs/1504.06066, 2015. 3, 7
- [30] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 2015. 3
- [31] M. A. Sadeghi and D. Forsyth. 30hz object detection with dpm v5. In *Computer Vision–ECCV 2014*, pages 65–79. Springer, 2014. 5, 6
- [32] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. *CoRR*, abs/1312.6229, 2013. 4, 5

参考文献

- [1] M. B. Blaschko 与 C. H. Lampert. 学习通过结构化输出回归定位物体。收录于 *Computer Vision–ECCV 2008*, 第2–15页。Springer, 2008年。4[2] L. Bourdev 与 J. Malik. 姿态基元: 使用3D人体姿态标注训练的身体部位检测器。收录于 *International Conference on Computer Vision (ICCV)*, 2009年。8[3] H. Cai, Q. Wu, T. Corradi, 与 P. Hall. 跨描绘问题: 识别艺术作品和照片中物体的计算机视觉算法。arXiv preprint arXiv:1505.00110, 2015年。7[4] N. Dalal 与 B. Triggs. 用于人体检测的方向梯度直方图。收录于 *Computer Vision and Pattern Recognition, 2005. CVPR 2005. IEEE Computer Society Conference on*, 第1卷, 第886–893页。IEEE, 2005年。4, 8[5] T. Dean, M. Ruzon, M. Segal, J. Shlens, S. Vijayanarasimhan, J. Yagnik 等。在单台机器上快速、准确地检测10万个物体类别。收录于 *Computer Vision and Pattern Recognition (CVPR), 2013 IEEE Conference on*, 第1814–1821页。IEEE, 2013年。5[6] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, 与 T. Darrell. Decaf: 用于通用视觉识别的深度卷积激活特征。arXiv preprint arXiv:1310.1531, 2013年。4[7] J. Dong, Q. Chen, S. Yan, 与 A. Yuille. 迈向统一的物体检测与语义分割。收录于 *Computer Vision–ECCV 2014*, 第299–314页。Springer, 2014年。7[8] D. Erhan, C. Szegedy, A. Toshev, 与 D. Anguelov. 使用深度神经网络的可扩展物体检测。收录于 *Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on*, 第2155–2162页。IEEE, 2014年。5, 6[9] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, 与 A. Zisserman. PASCAL视觉物体类别挑战: 回顾。 *International Journal of Computer Vision*, 111(1):98–136, 2015年1月。2[10] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, 与 D. Ramanan. 使用基于判别性训练部件模型的物体检测。 *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(9):1627–1645, 2010年。1, 4[11] S. Gidaris 与 N. Komodakis. 通过多区域和语义分割感知的CNN模型进行物体检测。 *CoRR*, abs/1505.01749, 2015年。7[12] S. Ginosar, D. Haas, T. Brown, 与 J. Malik. 在立体主义艺术中检测人物。收录于 *Computer Vision–ECCV 2014 Workshops*, 第101–116页。Springer, 2014年。7[13] R. Girshick, J. Donahue, T. Darrell, 与 J. Malik. 用于精确物体检测和语义分割的丰富特征层次结构。收录于 *Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on*, 第580–587页。IEEE, 2014年。1, 4, 7[14] R. B. Girshick. Fast R-CNN. *CoRR*, abs/1504.08083, 2015年。2, 5, 6, 7[15] S. Gould, T. Gao, 与 D. Kolle. 基于区域的分割与物体检测。收录于 *Advances in neural information processing systems*, 第655–663页, 2009年。4
- [16] B. Hariharan, P. Arbeláez, R. Girshick, J. Malik. 同步检测与分割。收录于 *Computer Vision–ECCV 2014*, 第297–312页。Springer, 2014年。7[17] K. He, X. Zhang, S. Ren, J. Sun. 用于视觉识别的深度卷积网络中的空间金字塔池化。arXiv preprint arXiv:1406.4729, 2014年。5[18] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, R. R. Salakhutdinov. 通过防止特征检测器的共适应来改进神经网络。arXiv preprint arXiv:1207.0580, 2012年。4[19] D. Hoiem, Y. Chodpathumwan, Q. Dai. 目标检测器中的错误诊断。收录于 *Computer Vision–ECCV 2012*, 第340–353页。Springer, 2012年。6[20] K. Lenc, A. Vedaldi. R-CNN 减去 R. arXiv preprint arXiv:1506.06981, 2015年。5, 6[21] R. Lienhart, J. Maydt. 用于快速目标检测的扩展类Haar特征集。收录于 *Image Processing. 2002. Proceedings. 2002 International Conference on*, 第1卷, 第1–900页。IEEE, 2002年。4[22] M. Lin, Q. Chen, S. Yan. 网络中的网络。 *CoRR*, abs/1312.4400, 2013年。2[23] D. G. Lowe. 基于局部尺度不变特征的目标识别。收录于 *Computer vision, 1999. The proceedings of the seventh IEEE international conference on*, 第2卷, 第1150–1157页。IEEE, 1999年。4[24] D. Mishkin. 模型在ImageNet 2012验证集上的准确率。 <https://github.com/BVLC/caffe/wiki/Models-accuracy-on-ImageNet-2012-val>. 访问于: 2015-10-2。3[25] C. P. Papageorgiou, M. Oren, T. Poggio. 目标检测的通用框架。收录于 *Computer vision, 1998. sixth international conference on*, 第555–562页。IEEE, 1998年。4[26] J. Redmon. Darknet: C语言中的开源神经网络。 <http://pjreddie.com/darknet/>, 2013–2016年。3[27] J. Redmon, A. Angelova. 使用卷积神经网络进行实时抓取检测。 *CoRR*, abs/1412.3128, 2014年。5[28] S. Ren, K. He, R. Girshick, J. Sun. Faster R-CNN: 利用区域提议网络实现实时目标检测。arXiv preprint arXiv:1506.01497, 2015年。5, 6, 7[29] S. Ren, K. He, R. B. Girshick, X. Zhang, J. Sun. 卷积特征图上的目标检测网络。 *CoRR*, abs/1504.06066, 2015年。3, 7[30] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, L. Fei-Fei. ImageNet大规模视觉识别挑战赛。 *International Journal of Computer Vision (IJCV)*, 2015年。3[31] M. A. Sadeghi, D. Forsyth. 使用DPM v5实现30Hz目标检测。收录于 *Computer Vision–ECCV 2014*, 第65–79页。Springer, 2014年。5, 6[32] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, Y. LeCun. OverFeat: 使用卷积网络进行集成识别、定位与检测。 *CoRR*, abs/1312.6229, 2013年。4, 5

- [33] Z. Shen and X. Xue. Do more dropouts in pool5 feature maps for better object detection. *arXiv preprint arXiv:1409.6911*, 2014. 7
- [34] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. *CoRR*, abs/1409.4842, 2014. 2
- [35] J. R. Uijlings, K. E. van de Sande, T. Gevers, and A. W. Smeulders. Selective search for object recognition. *International journal of computer vision*, 104(2):154–171, 2013. 4
- [36] P. Viola and M. Jones. Robust real-time object detection. *International Journal of Computer Vision*, 4:34–47, 2001. 4
- [37] P. Viola and M. J. Jones. Robust real-time face detection. *International journal of computer vision*, 57(2):137–154, 2004. 5
- [38] J. Yan, Z. Lei, L. Wen, and S. Z. Li. The fastest deformable part model for object detection. In *Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on*, pages 2497–2504. IEEE, 2014. 5, 6
- [39] C. L. Zitnick and P. Dollár. Edge boxes: Locating object proposals from edges. In *Computer Vision–ECCV 2014*, pages 391–405. Springer, 2014. 4

[33] Z. Shen and X. Xue. 在pool5特征图中使用更多dropout以改善目标检测。 *arXiv preprint arXiv:1409.6911*, 2014. 7[34] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. 通过卷积网络深入探索。 *CoRR*, abs/1409.4842, 2014. 2[35] J. R. Uijlings, K. E. van de Sande, T. Gevers, and A. W. Smeulders. 用于目标识别的选择性搜索。 *Inter- national journal of computer vision*, 104(2):154–171, 2013. 4[36] P. Viola and M. Jones. 鲁棒的实时目标检测。 *International Journal of Computer Vision*, 4:34–47, 2001. 4[37] P. Viola and M. J. Jones. 鲁棒的实时人脸检测。 *International journal of computer vision*, 57(2):137–154, 2004. 5[38] J. Yan, Z. Lei, L. Wen, and S. Z. Li. 用于目标检测的最快速可变形部件模型。 收录于 *Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on*, 第2497–2504页。 IEEE, 2014. 5, 6[39] C. L. Zitnick and P. Dollár. Edge boxes: 从边缘定位目标候选区域。 收录于 *Computer Vision–ECCV 2014*, 第391–405页。 Springer, 2014. 4